

PROJEKT IO

System wspomagający nauczanie studenta

Autorzy:

Mateusz Cegłowski

Jakub Kusek

Data: Maj 2026

1. Wstęp.....	3
2. Analiza wymagań.....	4
2.1. Aktorzy i ogólne przypadki użycia.....	4
2.2. Scenariusze przypadków użycia.....	5
2.3. Diagramy przejść i logika sterowania.....	7
3. Projekt systemu.....	10
3.1. Klasy, atrybut oraz metody.....	10
3.2. Działanie dynamiczne programu.....	12
4. Implementacja.....	14
4.1. Dziedziczenie i inicjalizacja obiektów.....	14
4.2. Asocjacja dwukierunkowa.....	14
4.3. Autoryzacja i rozpoznawanie roli.....	15
5. Testowanie.....	16
5.1. Testy jednostkowe.....	16
5.2. Testy Integracyjne.....	17
5.3. Testy Systemowe.....	19
6. Podsumowanie.....	22
6.1. Osiągnięte cele.....	22
6.2. Wnioski.....	22
6.3. Dalsze kroki.....	22

1. Wstęp

Niniejsza dokumentacja opisuje etapy tworzenia systemu. Wstępnie zostały zdefiniowane przypadki użycia, obrazujące kluczowe interakcje użytkowników systemu. Następnie opracowane zostały scenariusze oraz diagramy przepływu, które ilustrują logikę realizowanych procesów. Kolejny etap to projektowanie obiektowe, obejmuje ono identyfikację klas oraz stworzenie diagramów sekwencji. Wynikiem tego była implementacja gotowego kodu źródłowego. Ostatnim krokiem jest weryfikacja stworzonego oprogramowania poprzez przeprowadzenie przygotowanych przypadków testowych na poziomie jednostkowym, integracyjnym oraz systemowym.

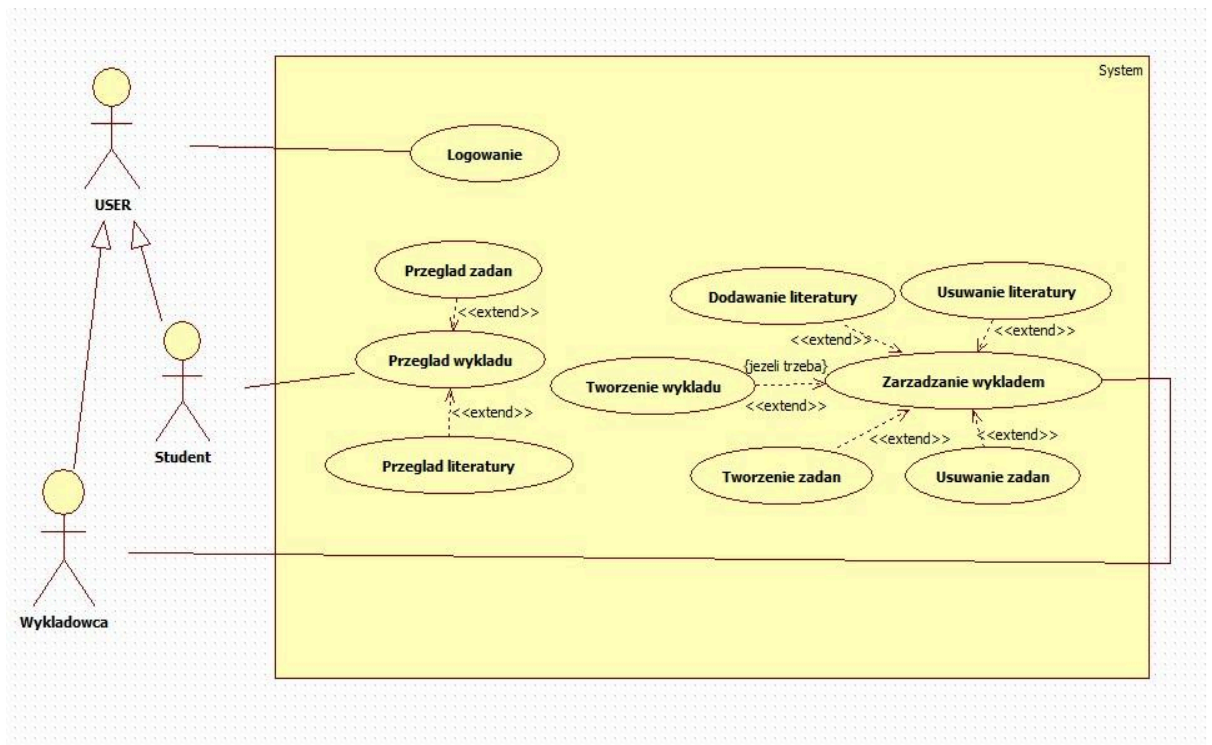
Głównym założeniem projektu jest stworzenie systemu wspomagającego studenta w nauce. System ma za zadanie modelować wirtualne środowisko uczelni i stworzyć połączenie między studentem a wykładowcą.

2. Analiza wymagań

Pierwszy etap projektowania systemu, polegający na analizie wymagań, zdefiniowaniu ról, i ogólnych przypadków użycia dla systemu.

2.1. Aktorzy i ogólne przypadki użycia

System został zaprojektowany z myślą o dwóch głównych aktorach: **Wykładowca** oraz **Student**, z których obaj dziedziczą wspólne cechy z bazowej klasy użytkownika. Przypadki użycia są przypisane do ich poszczególnych ról. Obaj aktorzy mogą logować się do systemu w celu autoryzacji. Po zalogowaniu, student może przeglądać materiały wewnątrz wykładu oraz dołączać do nowych zajęć, podczas gdy Wykładowca ma nad kursami pełną kontrolę, mogąc je tworzyć, ustawiać hasła oraz dodawać i usuwać materiały.



2.2. Scenariusze przypadków użycia

Scenariusze dla przypadków użycia wyglądają następująco:

Przypadek użycia	Logowanie
Scenariusz	Zalogowanie do systemu użytkownika
Warunki wstępne	Konto użytkownika istnieje.
Niezmienniki	Użytkownik chce się zalogować do systemu.
Opis	Użytkownik próbuje zalogować się do systemu. Użytkownik wpisuje login i hasło. System autoryzuje logowanie. System sprawdza poprawność logowania. Login i hasło podane przez użytkownika są poprawne. Użytkownik zostanie zalogowany do systemu.
Warunki końcowe	Użytkownik zostanie zalogowany do systemu.

Przypadek użycia	Tworzenie wykład
Scenariusz	Wykładowca tworzy nowy wykład
Warunki wstępne	Wykładowca jest zalogowany.
Niezmienniki	Wykładowca chce stworzyć nowy wykład.
Opis	Wykładowca wpisuje nazwę nowego kursu oraz wydziału na którym wykład ma się odbywać. System sprawdza czy wydział został wpisany poprawnie. System tworzy nowy wykład. Wykładowca wybiera czy dodać hasło do wykładu. System sprawdza czy wykładowca chce dodać hasło. Wykładowca wpisuje hasło do wykładu. System ustawia hasło dla wykładu. Wykład został utworzony.
Warunki końcowe	Wykładowca stworzył nowy wykład.

Przypadek użycia	Dołączenie do wykładu
Scenariusz	Student dołącza do wykładu
Warunki wstępne	Student jest zalogowany. Wykład istnieje.
Niezmienniki	Student chce dołączyć do wykładu
Opis	Student podaje nazwę wykładu. System wyszukuje wykład. Student podaje hasło do kursu. System porównuje hasło podane przez studenta z hasłem wykładu. Student zostaje dodany do wykładu.
Warunki końcowe	Student zostaje dodany do wykładu.

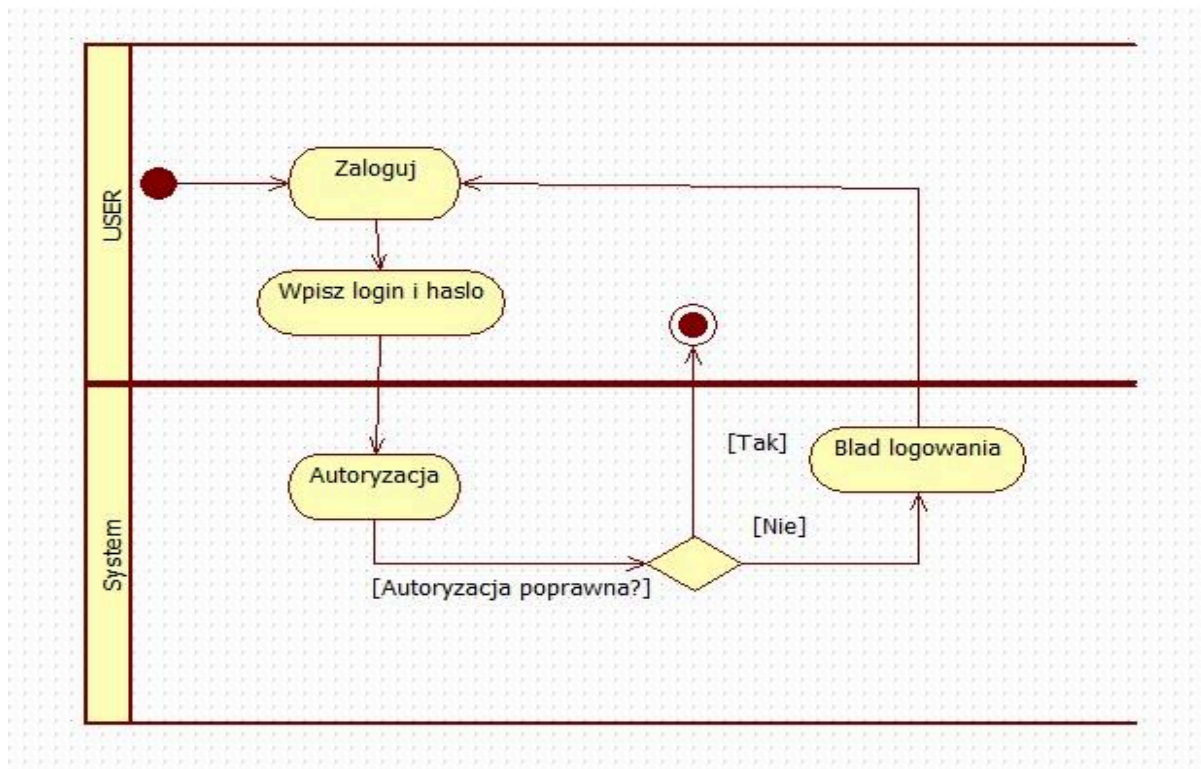
Przypadek użycia	Przeglądanie zadań
Scenariusz	Student przechodzi do przeglądania zadań.
Warunki wstępne	Student jest zalogowany. Wykład istnieje. Student jest zapisany do wykładu.
Niezmienniki	Student chce przejść do przeglądania zadań.
Opis	Student podaje nazwę wykładu. System wyszukuje wykład. System wyświetla listę zadań. Student przechodzi do przeglądania zadań.
Warunki końcowe	Student przechodzi do przeglądania zadań.

Przypadek użycia	Tworzenie zadań
Scenariusz	Wykładowca tworzy nowe zadanie.
Warunki wstępne	Wykładowca jest zalogowany. Wykład istnieje. Wykładowca jest zapisany do wykładu.
Niezmienniki	Wykładowca chce utworzyć nowe zadanie.
Opis	Wykładowca podaje nazwę wykładu. System wyszukuje wykład. Wykładowca podaje nazwę oraz typ zadania. System sprawdza czy zadanie już istnieje. Zadanie zostanie utworzone i dodane do wykładu.
Warunki końcowe	Zadanie zostanie utworzone.

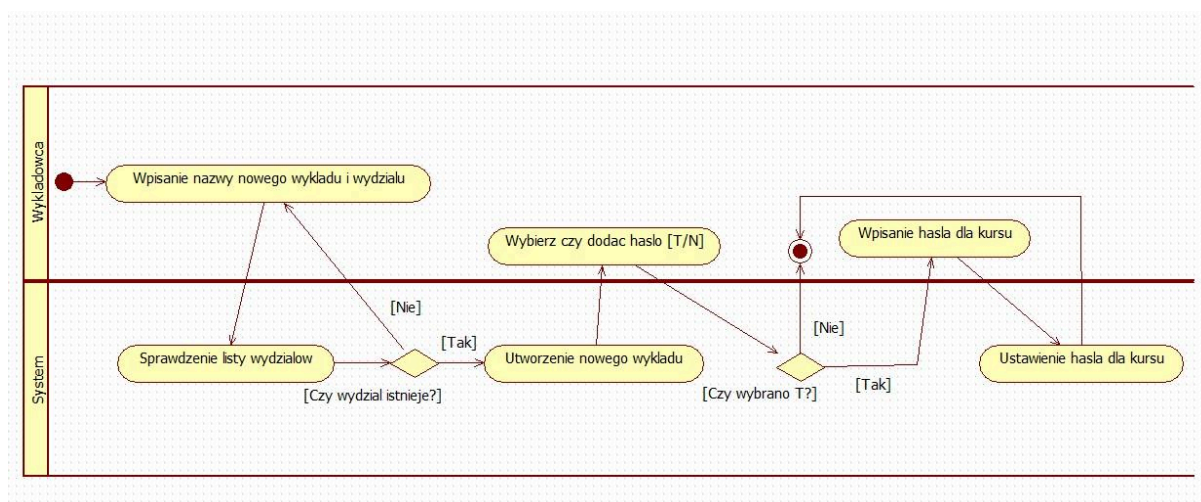
Przypadek użycia	Usuwanie zadania
Scenariusz	Wykładowca usuwa zadanie.
Warunki wstępne	Wykładowca jest zalogowany. Wykład istnieje. Zadanie istnieje. Wykładowca jest zapisany do wykładu.
Niezmienniki	Wykładowca chce usunąć zadanie.
Opis	Wykładowca podaje nazwę wykładu. System wyszukuje wykład. Wykładowca podaje nazwę zadania. System sprawdza czy zadanie istnieje. Zadanie zostanie usunięte.
Warunki końcowe	Zadanie zostanie usunięte.

2.3. Diagramy przejść i logika sterowania

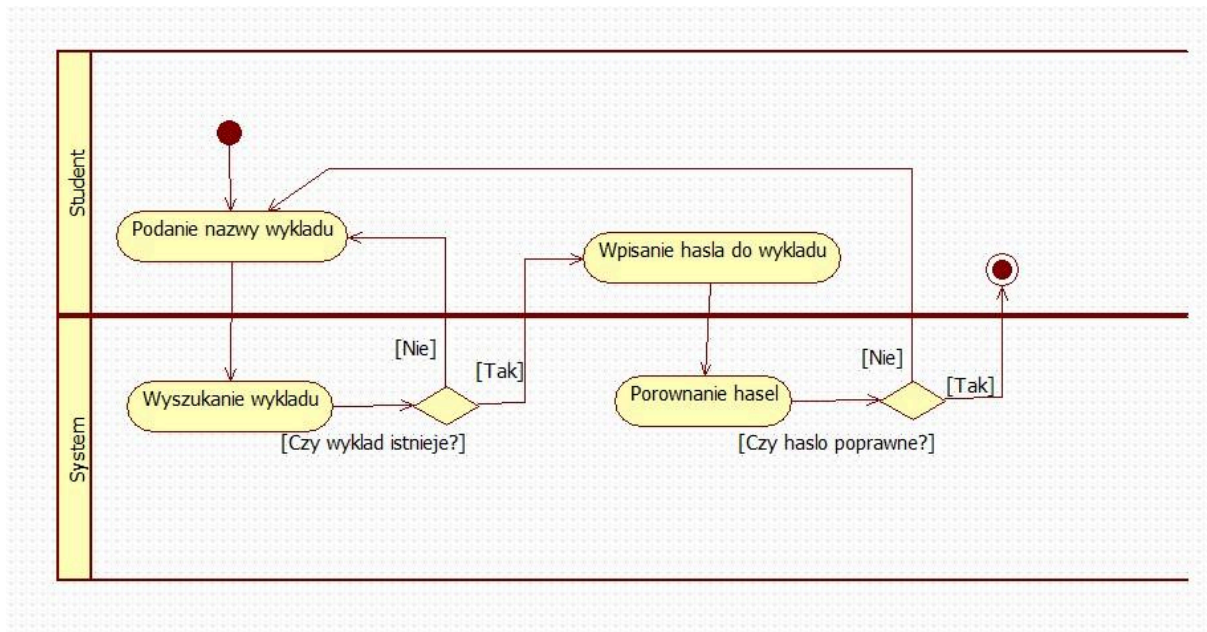
Logowanie do systemu:



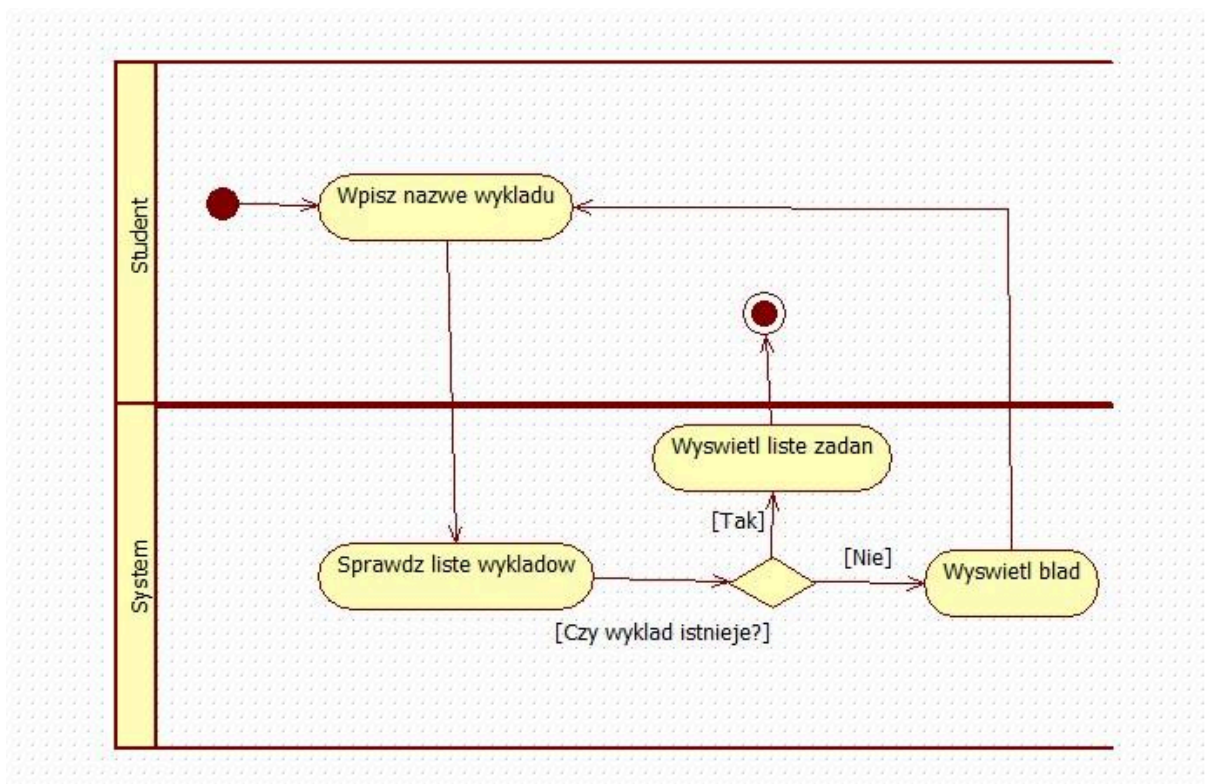
Tworzenie nowego wykładu przez wykładowcę:



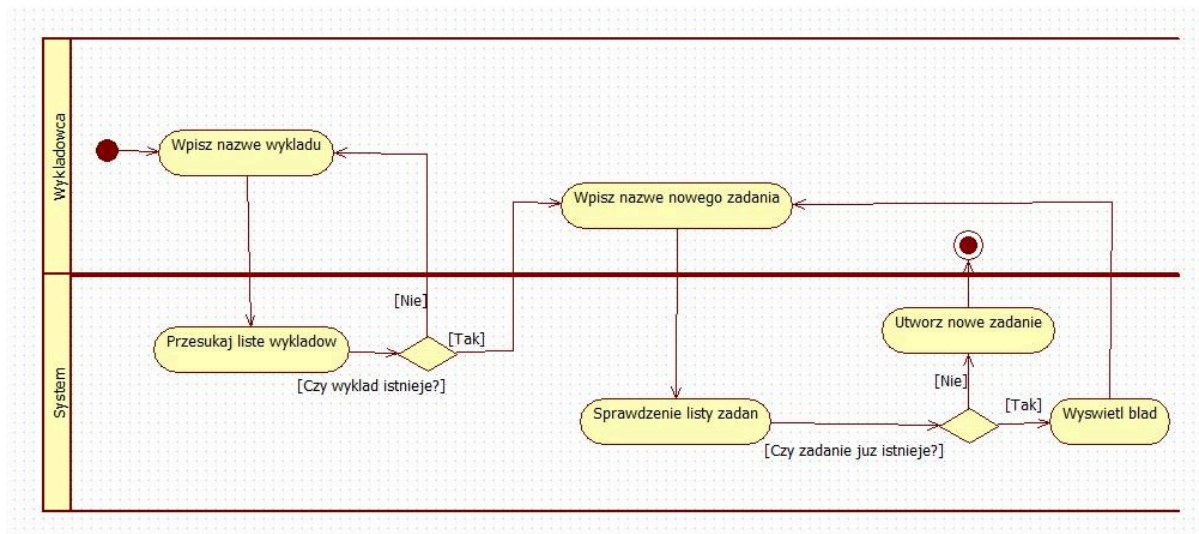
Dołączenie do wykładu przez studenta:



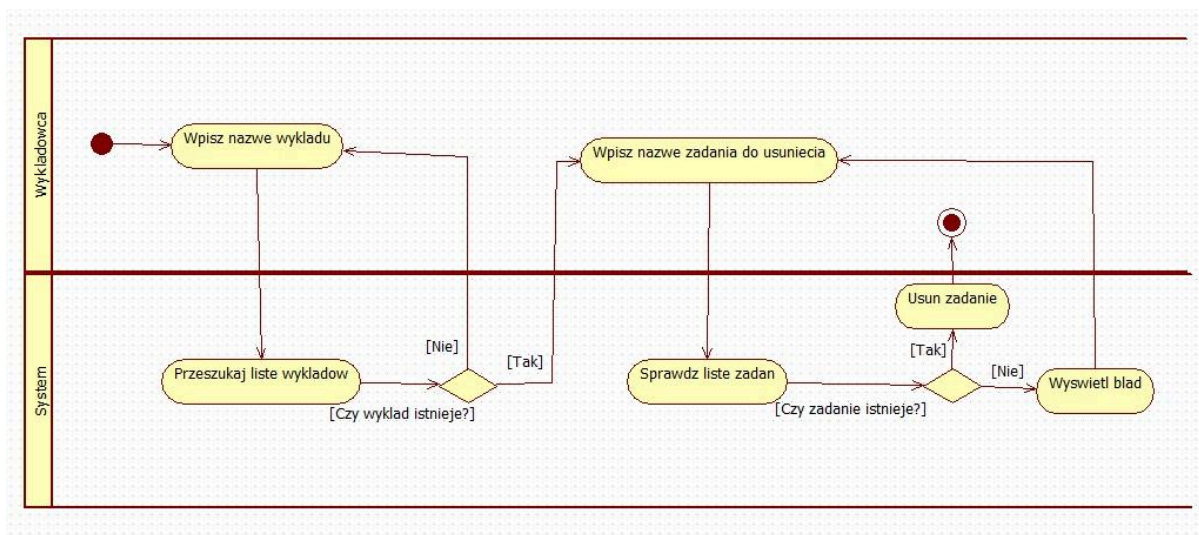
Przeglądanie zadań przez studenta:



Tworzenie zadań przez wykładowcę:



Usuwanie zadań przez wykładowcę:

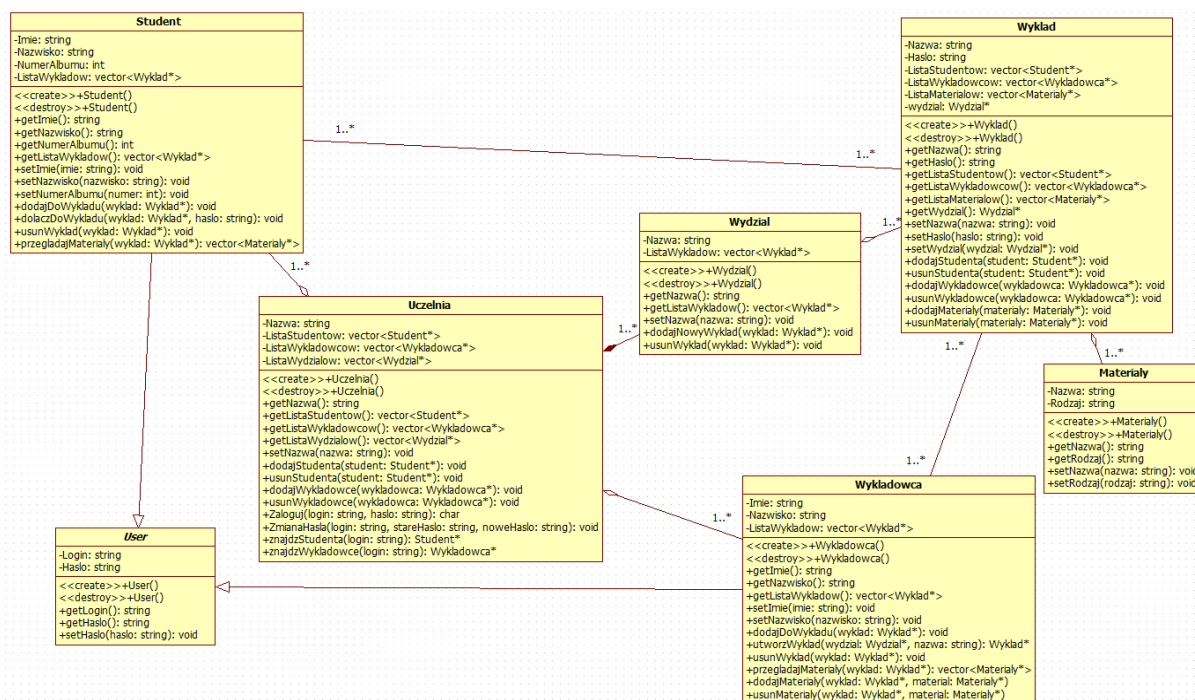


3. Projekt systemu

3.1. Klasy, atrybuty oraz metody.

Główne klasy w naszym projekcie to:

- **Student** - Przechowująca dane studenta takie jak imię, nazwisko, numer albumu oraz wykłady do których należy. Dziedziczy ona po klasie User.
- **Wykładowca** - Przechowująca dane wykładowcy takie jak imię, nazwisko, oraz wykłady do których należy. Dziedziczy ona po klasie User.
- **User** - Abstrakcyjna klasa bazowa dla Studenta i Wykładowcy. Przechowuje wspólne dane uwierzytelniające, takie jak oraz Login i Hasło a także udostępnia metody do ich weryfikacji i modyfikacji.
- **Uczelnia** - Klasa główna przechowująca nazwę uczelni, listy Studentów, Wykładowców i Wydziałów. Odpowiada za autoryzację użytkowników przez metodę logowania. Jej destruktor usuwa wszystkie pozostałe obiekty.
- **Wydział** - Klasa grupująca zajęcia. Przechowuje własną nazwę oraz dynamiczną listę przypisanych do danego wydziału obiektów klasy.
- **Wykład** - Reprezentuje konkretne zajęcia i przechowuje własną nazwę oraz opcjonalne hasło dostępu. Brak hasła jest domyślne i jest interpretowany jako ukrycie wykładu przed studentami. Integruje ze sobą inne obiekty, przechowując wektory przypisanych do zajęć Studentów, Wykładowców oraz Materiałów. Pośredniczy w połączeniu Student i Wykładowca.
- **Materiały** - Klasa reprezentująca pojedynczy zasób dydaktyczny. Przechowuje informacje o udostępnionym pliku, takie jak jego nazwa oraz format/rodzaj (np. "PDF", "Skrypt").

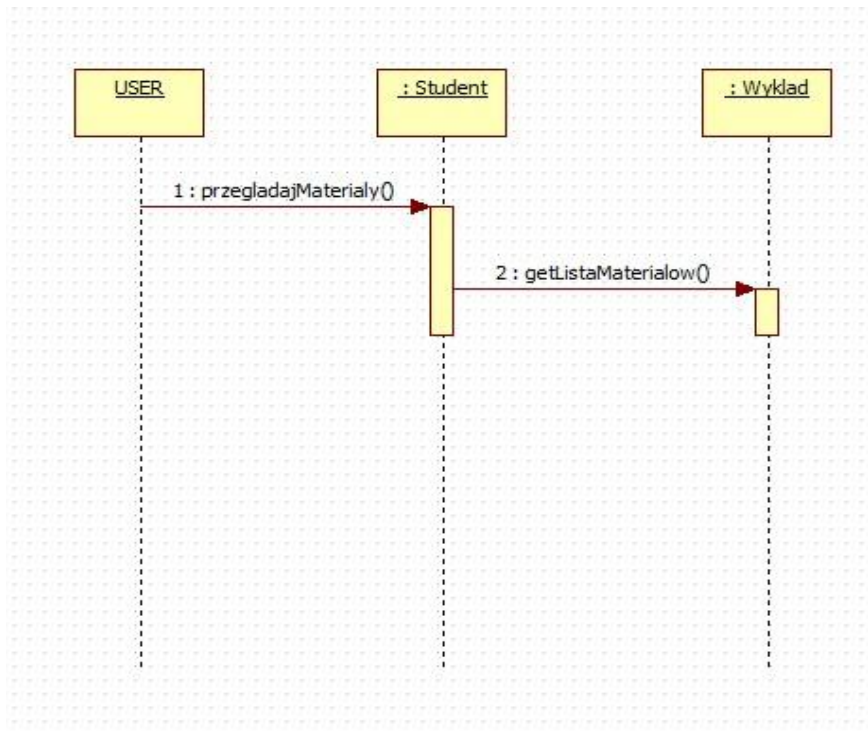


Poza metodami get i set w każdej z klasie występują:

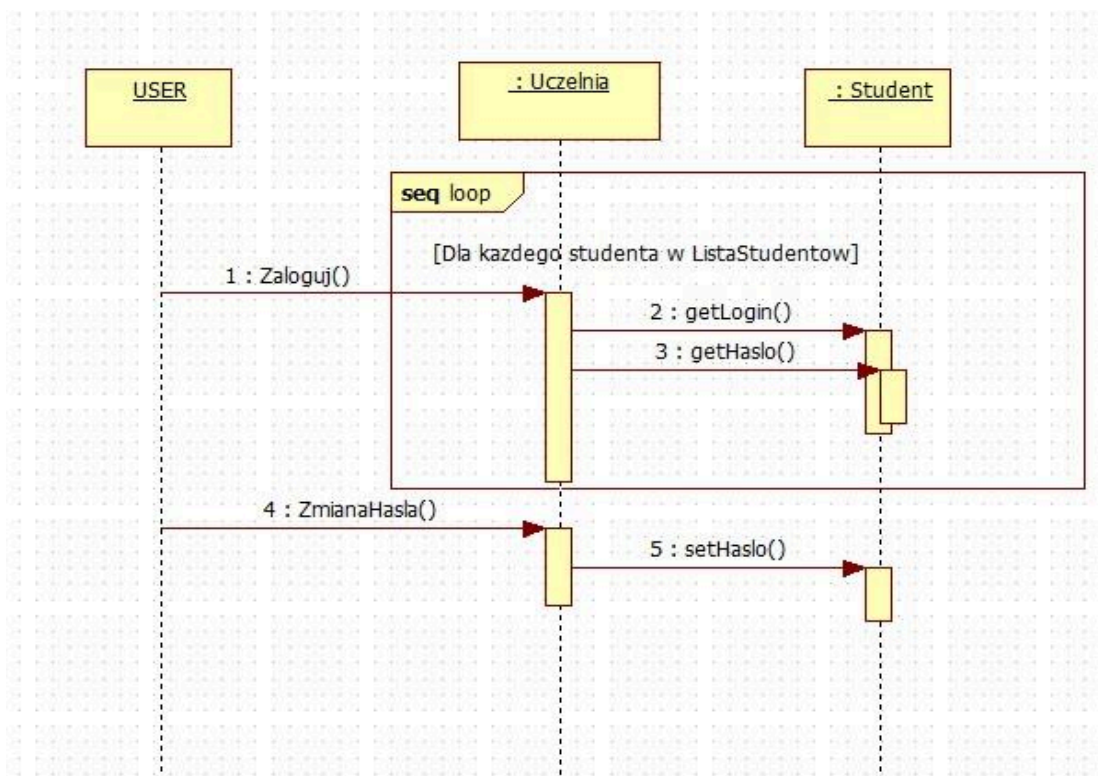
- **przełączajMateriały()** - dla klas Student i Wykładowca. Metoda poprzez obiekt wykładu w jej nagłówku odwołuje się do jego listy materiałów i zwraca jej kontener.
- **dolaczDoWykładu()** - Metoda klasy Student, umożliwiająca dołączenie studenta do wykładu z hasłem. Metoda porównuje hasło otrzymane w nagłówku z hasłem dla obiektu wykładu.
- **utworzWykład()** - Metoda klasy Wykładowca, pozwalająca utworzyć nowy obiekt Wykładu. Metoda wywołuje konstruktor Wykładu i przekazuje Wydział i nazwę. Po utworzeniu nowego obiektu wykładowca jest dołączany do jego listy.
- **Zaloguj()** - Metoda klasy Uczelnia służąca do logowania do systemu. Podane w nagłówku login i hasło są porównywane w pętli kolejno z danymi Studentów i Wykładowców. Metoda zwraca char który jest wykorzystywany w dalszej części programu. Jeżeli użytkownik zalogował się jako student zwracana jest litera 's', jako wykładowca zwracana jest litera 'w', albo litera 'f' jeżeli nie udało się zalogować.
- **dodajMateriały()** - Metoda klasy Wykładowca, dodająca obiekt materiałów do wykładu. Obiekt Materiały przekazywany w nagłówku metody jest dodawany do listy materiałów podanego Wykładu..
- **usuńMateriały()** - Metoda klasy Wykładowca, usuwająca obiekt materiałów z wykładu. Obiekt Materiały przekazywany w nagłówku metody jest usuwany z listy materiałów Wykładu a następnie wywołany jego destruktor.

3.2. Działanie dynamiczne programu.

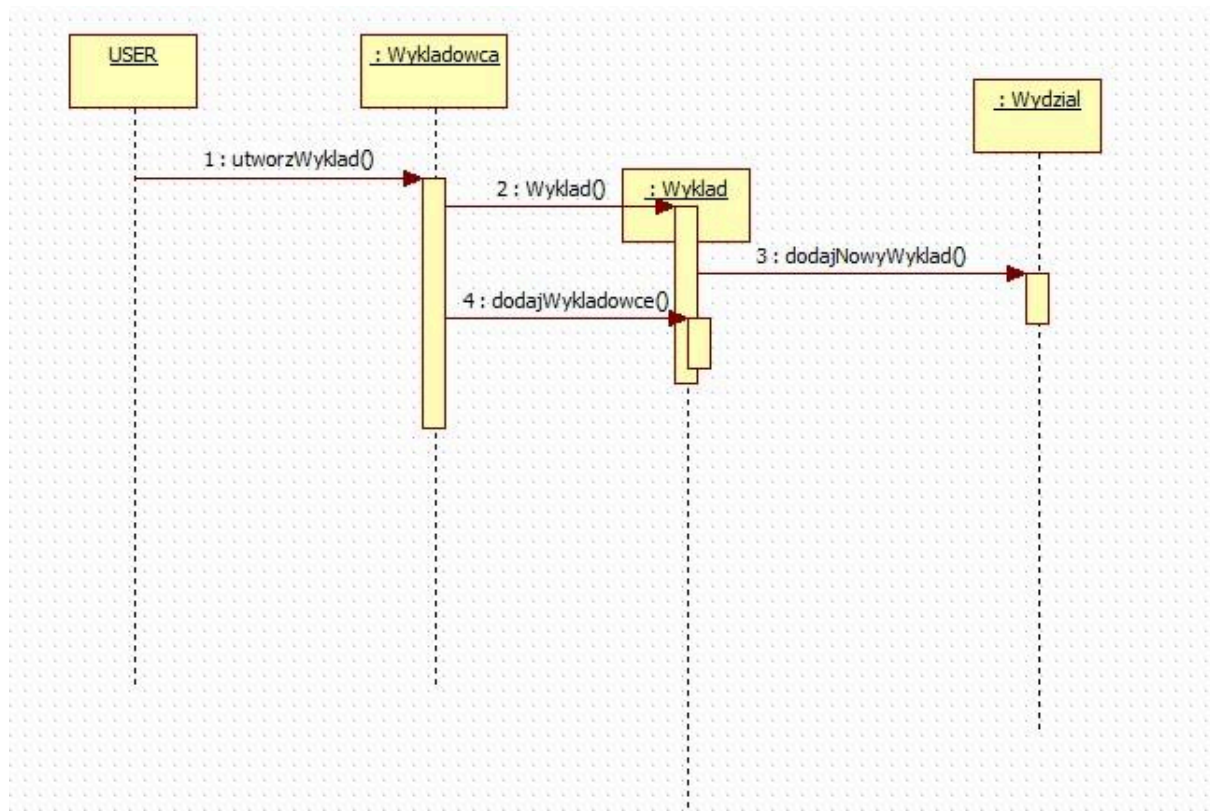
Wywołanie metody przeglądaj materiały:



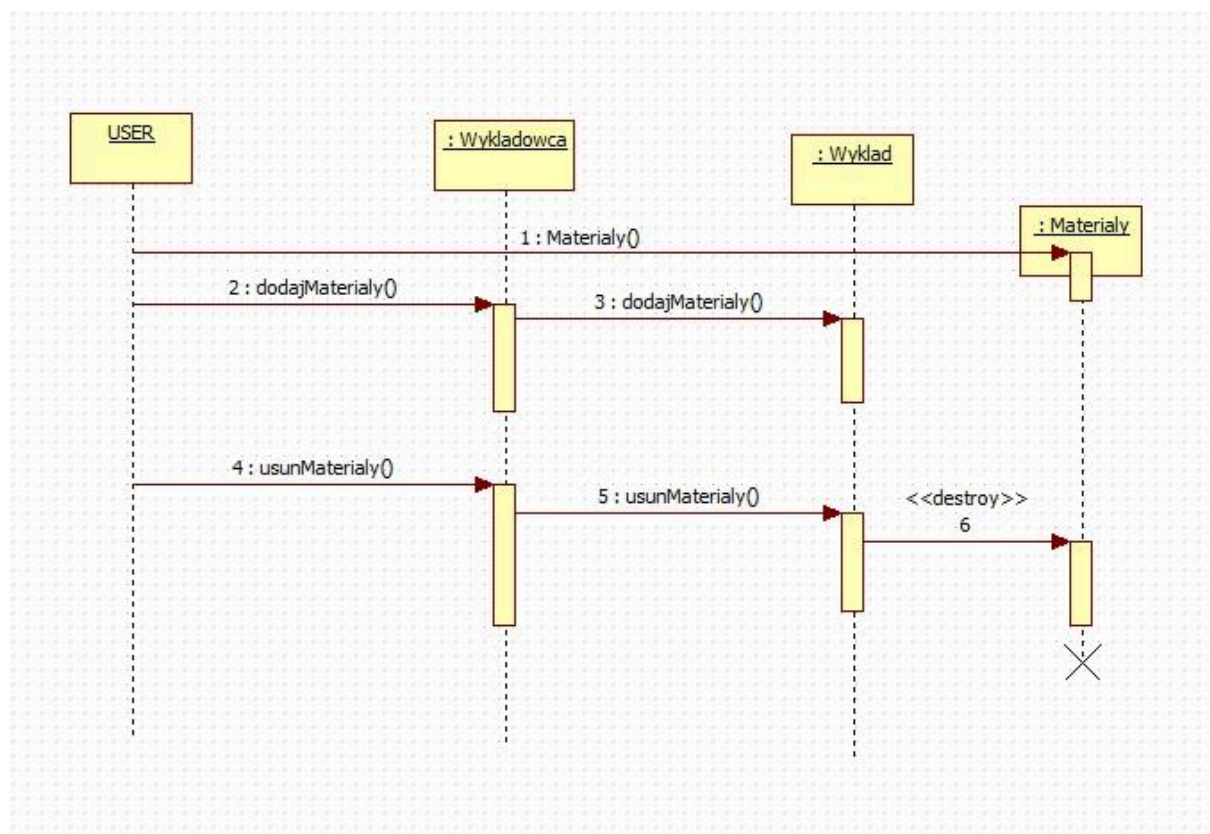
Autoryzacja i zmiana hasła w systemie:



Inicjalizacja i przypisywanie nowego wykładu:



Zarządzanie materiałami przez wykladowcę:



4. Implementacja

Etap implementacji polegał na fizycznym utworzeniu kodu źródłowego na podstawie przygotowanych wcześniej diagramów i specyfikacji. Projekt został zrealizowany w języku C++. Kod został podzielony na pliki nagłówkowe (.h) definiujące interfejsy oraz pliki źródłowe (.cpp) zawierające właściwą implementację logiki.

4.1. Dziedziczenie i inicjalizacja obiektów

Pierwszym etapem kodowania było utworzenie klas Student i Wykładowca, oraz wywołanie w ich konstruktorach konstruktor klasy bazowej. Obie klasy przyjmują hasło w nagłówku konstruktora, podobnie login dla wykładowcy natomiast login dla studenta stanowi jego numer albumu.

Fragment kodu klasy Student:

```
Student::Student(string imie, string nazwisko, int numerAlbumu, string haslo) :  
    User(to_string(numerAlbumu), haslo) {  
    this->Imie = imie;  
    this->Nazwisko = nazwisko;  
    this->NumerAlbumu = numerAlbumu;  
}
```

4.2. Asocjacja dwukierunkowa

Relacje między Studentem a Wykładem, oraz Wykładowcą a Wykładem są asocjacją dwukierunkową. Oba obiekty wiedzą o sobie i przechowują swoje wskaźniki w atrybutach. Dlatego każde dodanie do listy Studentów w Wykładzie powoduje wywołanie metody dodaj wykład dla obiektu Studenta.

Fragment kodu klasy Wykład:

```
void Wyklad::dodajStudenta(Student* student) {  
    auto it = find(this->ListaStudentow.begin(), this->ListaStudentow.end(),  
        student);  
    if (it == this->ListaStudentow.end()) {  
        this->ListaStudentow.push_back(student);  
        student->dodajDoWykladu(this);  
    }  
}
```

4.3. Autoryzacja i rozpoznawanie roli

Aby użytkownicy mogli korzystać z systemu, należało zaimplementować mechanizm logowania, sprawdzający poprawność danych, oraz identyfikujący rolę użytkownika (Student lub Wykładowca).

Fragment kodu klasy Uczelnia:

```
char Uczelnia::Zaloguj(string login, string haslo) {
    for (Student* s : this->ListaStudentow) {
        if (s->getLogin() == login && s->getHaslo() == haslo) {
            cout << "Zalogowano pomyslnie jako Student: " << login << endl;
            return 's';
        }
    }

    for (Wykladowca* w : this->ListaWykladowcow) {
        if (w->getLogin() == login && w->getHaslo() == haslo) {
            cout << "Zalogowano pomyslnie jako Wykladowca: " << login << endl;
            return 'w';
        }
    }

    cout << "Bład logowania."<< endl;
    return 'f';
}
```

5. Testowanie

Ostatnim etapem był proces testowania kodu, w celu sprawdzenia jego prawidłowego działania.

5.1. Testy jednostkowe

Pierwsze proste testy działające w obrębie pojedynczych metod. Mają na celu pokazanie ich prawidłowego działania. Testowane przez nas metody to m.in. dodanie do listy, usuwanie z listy, czy sprawdzenie poprawnego działania metody zmień hasło.

Test metody zmień hasło:

```
bool zmianaHasla(){
    Wydzial* wydz = new Wydzial("Wydzial Matematyki");
    vector<Wydzial*> wektor;
    wektor.push_back(wydz);

    Uczelnia* ucz = new Uczelnia("Uczelnia",wektor);
    Student* st1 = new Student("Imie", "Nazwisko", 1234, "haslo");

    ucz->dodajStudenta(st1);
    ucz->ZmianaHasla("1234", "haslo", "haslo1");

    bool result = st1->getHaslo() == "haslo1";

    delete ucz;
    return result;
}
```

Test metody dodającej do listy wykładu:

```
bool dodanieDoListyWykladowWydzial(){
    Wydzial* wydz = new Wydzial("Wydzial Matematyki");
    Wyklad* wyk = new Wyklad("wyklad1", wydz);

    bool result = wydz->getListaWykladow().size() == 1;

    delete wydz;
    return result;
}
```

Test metody usuwania wykładu z listy:

```

bool usuwanieZListyWykladowWydzial(){
    Wydzial* wydz = new Wydzial("Wydzial");
    Wyklad* wyk = new Wyklad("wyklad1", wydz);

    wydz->usunWyklad(wyk);

    bool result = wydz->getListaWykladow().size() == 0;

    delete wydz;
    return result;
}

```

5.2. Testy Integracyjne

Sprawdzenie działania wszystkich klas i metod sprawdzaliśmy modelem wielkiego wybuchu, czyli utworzeniem niewielkiego fragmentu programu testującego poszczególne metody.

Fragment kodu:

```

cout << "=== 1. INICJALIZACJA SYSTEMU ===" << endl;
// Wydziały
Wydzial* wydzInformatyki = new Wydzial("Wydział Informatyki");
Wydzial* wydzMatematyki = new Wydzial("Wydział Matematyki");

vector<Wydzial*> wszystkieWydzialy;
wszystkieWydzialy.push_back(wydzInformatyki);
wszystkieWydzialy.push_back(wydzMatematyki);

// Uczelnia
Uczelnia* uczelnia = new Uczelnia("Uniwersytet Marii Curie-Skłodowskiej",
wszystkieWydzialy);
cout << "Utworzono uczelnie: " << uczelnia->getNazwa() << endl;

// Studenci
Student* s1 = new Student("Jan", "Kowalski", 100111, "haslo1");
Student* s2 = new Student("Anna", "Nowak", 100222, "haslo2");
uczelnia->dodajStudenta(s1);
uczelnia->dodajStudenta(s2);

// Wykładowcy
Wykladowca* w1 = new Wykladowca("Alan", "Turing", "aturing", "tajne123");
Wykladowca* w2 = new Wykladowca("Ada", "Lovelace", "alovelace", "tajne456");
uczelnia->dodajWykladowce(w1);
uczelnia->dodajWykladowce(w2);

```

```

cout << "Zarejestrowano studentow i wyklowcow.\n" << endl;

cout << "=== 2. TWORZENIE WYKLADOW ===" << endl;
Wyklad* wykInf1 = w1->utworzWyklad(wydzInformatyki, "Programowanie C++");
wykInf1->setHaslo("cpp2026");
cout << "Wykladowca " << w1->getNazwisko() << " utworzyl wyklad: " <<
wykInf1->getNazwa() << " (Z HASLEM)." << endl;

Wyklad* wykMat1 = w2->utworzWyklad(wydzMatematyki, "Analiza Matematyczna");
cout << "Wykladowca " << w2->getNazwisko() << " utworzyl wyklad: " <<
wykMat1->getNazwa() << " (BEZ HASLA)." << endl;

cout << "\n=== 3. DODAWANIE MATERIALOW ===" << endl;
w1->dodajMaterialy(wykInf1, new Materialy("Wstep do C++", "Prezentacja PDF"));
w1->dodajMaterialy(wykInf1, new Materialy("Zadanie 1: Klasy", "Plik TXT"));
w1->dodajMaterialy(wykInf1, new Materialy("Kolokwium probne", "Skrypt .sh"));

w2->dodajMaterialy(wykMat1, new Materialy("Calki - Teoria", "Dokument PDF"));
w2->dodajMaterialy(wykMat1, new Materialy("Zestaw zadan", "Plik DOCX"));

cout << "Pomyslnie przypisano materialy do kursow.\n" << endl;
cout << "=== 4. DOLACZANIE DO WYKLADOW ===" << endl;
cout << "Student " << s1->getImie() << " probuje dolaczyc do 'Programowanie C++'
podajac zle haslo:" << endl;
s1->dolaczDoWykladu(wykInf1, "zle_haslo");

cout << "\nStudent " << s1->getImie() << " probuje dolaczyc podajac prawidlowe haslo:"
<< endl;
s1->dolaczDoWykladu(wykInf1, "cpp2026");

wykMat1->dodajStudenta(s2);
cout << "Studentka " << s2->getImie() << " zostala recznie zapisana na wyklad z
Analizy.\n" << endl;

cout << "=== 5. PREZENTACJA WYNIKOW ===" << endl;
cout << "Materialy dostepne dla studenta " << s1->getImie() << " na wykladzie " <<
wykInf1->getNazwa() << ":" << endl;
for(Materialy* mat : s1->przegladajMaterialy(wykInf1)){
    cout << " -> " << mat->getNazwa() << " [" << mat->getRodzaj() << "]" << endl;
}

```

5.3. Testy Systemowe

Ostateczne testowanie programu przedstawiające jego rzeczywiste działanie. Utworzyliśmy kod symulujący działanie panelu sterowania. Po zalogowaniu użytkownik otrzymuje różne warianty działania zależne od roli.

Jako student może przeglądać materiały, dołączyć do kursu lub zmienić swoje hasło.

Wykładowca może dodatkowo dodawać i usuwać materiały w danym wykładzie, tworzyć nowe wykłady i zmieniać hasła dla swoich wykładów.

Część kodu obsługującego panel studenta:

```
case 's':{
    zalogowanyStudent = uczelnia1->znajdzStudenta(login);
    while(1){
        cout << "\n--- MENU STUDENTA ---" << endl;
        cout << "Wybierz numer:" << endl;
        cout << "1.Przeglądaj materiały" << endl;
        cout << "2.Zmien hasło" << endl;
        cout << "3.Dolacz do wykladu" << endl;
        cout << "4.Wyloguj" << endl;
        cin >> decyzja;

        switch (decyzja) {
            case '1':{
                cout << "Wybierz wyklad: " << endl;

                for(Wykład* w :zalogowanyStudent->getListaWykladow()){
                    cout << w->getNazwa() << endl;
                }

                string nazwaWykladu;
                bool znaleziono = false;
                std::cin.ignore();
                getline(cin ,nazwaWykladu);

                for(Wykład* w :zalogowanyStudent->getListaWykladow()){
                    if(nazwaWykladu == w->getNazwa()){
                        znaleziono = true;
                        for(Materiały* mat:
zalogowanyStudent->przegladaJMaterialy(w)){
                            cout << mat->getNazwa() << endl;
                        }
                    }
                }
            }
        }
    }
}
```

```

        }
    }
}
    if (!znaleziono){
        cout << "Nie znaleziono danego wykładu." << endl;
    }
    break;
}
case '2':{
    string starehaslo, nowehaslo;

    cout << "Wpisz stare haslo: " << endl;
    cin >> starehaslo;
    cout << "Wpisz nowe haslo: " << endl;
    cin >> nowehaslo;

    uczelnia1->ZmianaHasla(login, starehaslo ,nowehaslo);

    break;
}
case '3':{
    cout << "Dostepne wykłady:" << endl;
    bool saDostepneWyklady = false;

    for (Wydzial* wydziel : uczelnia1->getListaWydzialow()) {
        for (Wyklad* w : wydziel->getListaWykladow()) {
            if (!w->getHaslo().empty()) {
                auto mojeWyklady = zalogowanyStudent->getListaWykladow();
                if (find(mojeWyklady.begin(), mojeWyklady.end(), w) == mojeWyklady.end())
                    cout << "- " << w->getNazwa() << " (Wydział: "
                        << wydziel->getNazwa() << ")" << endl;
                saDostepneWyklady = true;
            }
        }
    }
}

if (!saDostepneWyklady) {
    cout << "Brak nowych wykładów, do których możesz dołączyć." << endl;
    break;
}
}

```

```

cout << "Wpisz nazwe wykladu, do ktorego chcesz dolaczyc: " << endl;

string nazwaWyk;
cin.ignore();
getline(cin, nazwaWyk);

Wyklad* wybranyWyklad = nullptr;
for (Wydzial* wydz : uczelnia1->getListaWydzialow()) {
    for (Wyklad* w : wydz->getListaWykladow()) {
        if (w->getNazwa() == nazwaWyk && !w->getHaslo().empty()) {
            wybranyWyklad = w;
            break;
        }
    }
    if (wybranyWyklad) break;
}

if (wybranyWyklad != nullptr) {
    cout << "Wpisz haslo do wykladu: " << endl;
    string podaneHasloWykladu;
    getline(cin, podaneHasloWykladu);

    zalogowanyStudent->dolaczDoWykladu(wybranyWyklad, podaneHasloWykladu);
} else {
    cout << "Wyklad nie istnieje lub jest niewidoczny (brak hasla)." << endl;
}
break;
}

case '4':{
    cout << "Wylogowano" << endl;
    return;
}
}
break;
}

```

6. Podsumowanie

6.1. Osiągnięte cele

W ramach zrealizowanego projektu udało się z powodzeniem zaprojektować i zaimplementować kluczowy fragment systemu wspomagającego studenta w nauce. Skutecznie zamodelowano wycinek wirtualnego środowiska uczelni, tworząc podstawy do komunikacji i wymiany informacji między studentem a wykładowcą. Przejście przez pełny cykl tworzenia oprogramowania zaowocowało spójnym i funkcjonalnym rozwiązaniem. Przeprowadzone pomyślnie testy potwierdziły poprawność działania aplikacji i jej zgodność z pierwotnymi wymaganiami.

6.2. Wnioski

Realizacja poszczególnych etapów udowodniła, że przygotowanie fazy analityczno-projektowej (szczegółowe scenariusze, diagramy przepływu) znacznie usprawnia właściwą implementację. Podejście obiektowe idealnie sprawdziło się przy odwzorowywaniu rzeczywistego połączenia między studentem, a wykładowcą w kodzie. Ponadto, kompleksowe podejście do weryfikacji oprogramowania okazało się kluczowe, testowanie na różnych poziomach integracji pozwoliło na szybkie zidentyfikowanie i wyeliminowanie potencjalnych błędów logiki systemu przed jego ostatecznym wdrożeniem.

6.3. Dalsze kroki

Zgodnie z przyjętym na początku iteracyjnym modelem rozwoju oprogramowania, przygotowany fragment stanowi solidny fundament do dalszej rozbudowy. W kolejnych iteracjach można rozwinąć ten system poprzez następujące działania:

Rozszerzenie funkcjonalności: Wzbogacenie systemu o nowe moduły, takie jak elektroniczny indeks ocen oraz harmonogram zajęć.

Rozwój interfejsu graficznego: Stworzenie intuicyjnego, przyjaznego dla użytkownika front-endu, który ułatwi obsługę systemu.

Rozbudowa struktury aktorów: Dodanie ról administracyjnych (np. dziekanat, administrator systemu) oraz mechanizmów zarządzania całymi grupami zajęciowymi.

Zabezpieczenia i optymalizacja: Rozbudowa istniejącego mechanizmu logowania o uwierzytelnianie dwuetapowe oraz przeprowadzenie testów wydajnościowych, przygotowujących aplikację na płynną obsługę znacznie większej liczby jednoczesnych użytkowników.